



事务 (Transaction)

“事务如誓言，要么彻底履行，要么彻底作废，
容不得半途而废。”

井明

数据科学与计算机学院

jingming@sdu.edu.cn

2025年春



山东女子学院
Shandong Women's University

课程导入案例：银行转账的意外

场景设定：

假设你在网上银行进行一次转账操作，把 1000元从你的一张卡转到另一张卡上。这个操作在系统中可能被拆分为两个步骤：

1. 从A账户扣除1000元；
2. 向B账户增加1000元。

思考：

1. 如果第一步成功执行了，系统从你的A账户里扣了1000元，但在执行第二步时，突然系统宕机了，导致B账户没有收到这笔钱。请问，这笔钱去哪了？
2. 我们希望这两个步骤是什么关系？如果一个成功了，另一个失败了，我们希望系统该怎么办？

结论： 要么都执行，要么都不执行。

““像这样需要操作要么全部成功，要么全部取消的场景。——数据库的事务机制（Transaction）。”**”**

事务的基本概念

事务 (Transaction) 定义:

一组对数据库的操作，这些操作要么全部执行，要么全部不执行，是数据库操作的最小工作单元。

为什么需要事务:

- 保证数据一致性
- 应对并发操作
- 处理系统故障等异常情况

事务的四大特性（ACID）

- 原子性（**A**tomicity）：
 - 事务中的所有操作要么全部完成，要么全部不执行。
- 一致性（**C**onsistency）：
 - 事务执行前后，数据库从一个一致状态转变为另一个一致状态。
- 隔离性（**I**solation）：
 - 并发事务之间互不干扰。
- 持久性（**D**urability）：
 - 一旦事务提交，其对数据库的修改是永久性的。

原子性 (Atomicity)

💡 概念回顾:

原子性 (Atomicity) 是指事务中的一系列操作，要么全部执行成功，要么全部不执行，不允许只执行其中一部分。

✅ 案例解释：银行转账

👛 业务场景：

用户小李从A账户转账1000元到B账户，数据库操作如下：

1. A账户余额 -1000
2. B账户余额 +1000

🔧 系统内部操作：

```
BEGIN;  
UPDATE account SET balance = balance - 1000 WHERE id = 'A';  
UPDATE account SET balance = balance + 1000 WHERE id = 'B';  
COMMIT;
```

✗异常情况:

如果系统在执行完第一条 UPDATE 后, 突然断电或程序崩溃, 第二条语句未执行, 那么:

- A账户少了1000元
- B账户没有增加1000元
- 资金“消失”了, 数据库进入了不一致状态

🛡️原子性的作用:

使用事务包裹整个操作后, 如果系统中途崩溃, 数据库能够自动回滚 (ROLLBACK), 让A账户恢复原状 (不扣钱), 保证事务要么全部成功, 要么全部失败, 维护了数据的一致性和安全性。

一致性（Consistency）

一致性（Consistency）：

事务执行前后，数据库都必须处于一个合法的、一致的状态。

“ 也就是说，事务的执行不能破坏数据库中数据之间定义好的约束、规则或业务逻辑。

✅ 案例解释：银行账户约束

👛 业务背景：

某银行规定：所有账户余额不能为负数，即系统中有一个约束：

```
CHECK (balance >= 0)
```

💰 业务操作场景：

用户小王准备从账户 A 中转出 5000 元，但此时账户余额只有 3000 元。

✗问题情境:

系统不检查余额直接执行扣款:

```
UPDATE account SET balance = balance - 5000 WHERE id = 'A';
```

执行后账户A变成 -2000 元, 违反了数据一致性规则 ($\text{balance} \geq 0$)。

🛡一致性的保护方式:

- 通过数据库约束 (如 CHECK、FOREIGN KEY、NOT NULL) 进行强制校验;
- 通过事务机制, 如果执行事务导致一致性被破坏, 数据库系统会自动回滚, 恢复到事务开始前的一致状态;
- 通过应用逻辑检查: 在执行前判断是否满足业务逻辑 (如余额足够)。

总结：

一致性确保了数据库始终符合“规则”或“业务逻辑”，即便系统发生异常或多个事务并发执行，也不能允许“逻辑错误”或“脏数据”进入数据库。

练习：

设计一个简单的账户转账系统，并加入如下约束：

- 每个账户余额不能为负；
- 所有账户总余额恒为某个固定值；
- 模拟事务前后一致性是否被破坏。

隔离性 (Isolation)

隔离性 定义:

多个事务并发执行时, 每个事务的执行都不应被其他事务干扰, 就像每个事务“在自己的世界中运行”一样。

“ 但完全隔离性能开销大, 所以数据库提供了不同的隔离级别来在性能与隔离性之间权衡。

✅ 案例解释：商品库存读取（引发“脏读”）

👛 业务背景：

电商平台中，一位用户正在购买商品，系统需要读取库存数量。

💻 两个事务并发执行：

1. 事务A（运营人员在后台修改库存）：

```
BEGIN;  
UPDATE product SET stock = stock - 10 WHERE id = 1; -- 修改库存  
-- 尚未 COMMIT
```

2. 事务B（前台用户查询库存）：

```
SELECT stock FROM product WHERE id = 1;
```

✗问题情境：脏读（Dirty Read）

如果数据库的隔离级别是 Read Uncommitted，事务B读到了事务A尚未提交的数据（可能还会回滚），这就叫脏读。

假如事务A最终失败并回滚了，事务B所读的库存数据是临时、不真实的，数据库中根本没有那种状态。

🎮其他并发异常与隔离性解释:

问题	描述	出现条件
脏读	读取了另一个未提交事务修改的数据	Read Uncommitted
不可重复读	同一个查询在同一事务中两次执行，读到不同结果	Read Committed
幻读	同一查询条件下，第二次读取发现插入了“新行”	Repeatable Read 以下隔离级别

 演示方式:

可以用如下方式在 MySQL 或 PostgreSQL 上演示:

1. 开启两个客户端连接;
2. 在一个连接中执行未提交的更新;
3. 在另一个连接中执行读取语句;
4. 配置不同的隔离级别进行观察。

```
-- 设置隔离级别  
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

总结

“在并发世界中，隔离性就像‘独立小黑屋’，每个事务只能看见自己已确定的世界，不能偷看别人还没完成的操作。”

💡 持久性（Durability）：

持久性指的是：一旦事务提交，它对数据库所做的修改就是永久性的，即使系统发生故障也不会丢失。

“ 持久性由数据库的日志机制（如WAL、重做日志）和存储机制保证。

✅ 案例解释：在线商城提交订单

👛 业务场景：

小张在某电商平台下单，系统在后台执行以下事务操作：

1. 生成订单记录；
2. 扣除商品库存；
3. 写入支付信息。

执行完成后系统执行：

```
COMMIT;
```

此时返回“下单成功”提示给用户。

❌ 问题情境：提交后断电

在用户看到“下单成功”提示的一瞬间，服务器突然宕机（如断电、崩溃）。

持久性的作用：

因为事务已提交，数据库会确保：

- 即便系统崩溃重启，订单、库存、支付信息的记录仍然存在；
- 恢复过程会通过**重做日志（REDO log）**将已提交但尚未落盘的修改重新执行一次，确保数据不会“丢失”；
- 用户下单的结果对系统来说是不可逆、永久生效的。

技术支撑：

数据库通过以下方式保障持久性：

- WAL机制（Write-Ahead Logging）：日志先于数据写入磁盘；
- 双写机制（如 InnoDB 的 redo log + data page）
- 崩溃恢复模块：如 MySQL 启动时自动 replay redo log

总结：

“提交之后，就像盖了章、入了账本，不论机器怎么崩，数据库也不会赖账！”

模拟演示：

布置一个断电恢复实验，例如：

- 在事务提交前关闭数据库模拟宕机，观察数据状态；
- 提交后再关闭数据库，重启后验证数据是否持久。

事务的生命周期

1. 开始事务 (BEGIN / START TRANSACTION)
2. 执行操作 (如 SELECT、INSERT、UPDATE、DELETE)
3. 提交事务 (COMMIT)
4. 回滚事务 (ROLLBACK)

并发控制

- 并发问题（事务未正确隔离会出现的问题）：
 - 脏读（Dirty Read）
 - 不可重复读（Non-repeatable Read）
 - 幻读（Phantom Read）
- 隔离级别（Isolation Levels）（SQL标准定义）：
 - i. Read Uncommitted（读未提交）
 - ii. Read Committed（读已提交）
 - iii. Repeatable Read（可重复读）
 - iv. Serializable（可串行化）

“ check session transaction level (mysql8+): `SELECT @@transaction_ISOLATION;`

“ check global transaction level (mysql8+): `SELECT @@global.transaction_ISOLATION;`

脏读 (Dirty Read)

💡 脏读定义：

一个事务读取了另一个未提交事务所修改的数据。

如果那个事务后来回滚了，那么读取到的数据就是“无效的、脏的”。

✅ 案例解释：库存查询读到了“假数据”

👛 业务场景：

商品库存系统，库存记录如下：

商品ID	名称	库存
1	手机A	100

并发执行的两个事务：

1. 事务A（运营人员在后台更新库存）：

```
BEGIN;  
UPDATE product SET stock = stock - 10 WHERE id = 1;  -- 暂时扣除10个库存  
-- COMMIT 尚未执行
```

2. 事务B（前台用户读取库存）：

```
-- 在默认或较低隔离级别下  
SELECT stock FROM product WHERE id = 1;  
-- 结果返回：90
```

3. 接着：

- 事务A由于检测到系统异常，执行 ROLLBACK;
- 实际上库存还是100，但事务B已经读到了错误的库存90

✖ 问题总结:

- 事务B读到了事务A“尚未提交”的修改
- 如果事务A后来回滚，事务B读到的数据就从未真正存在过
- 这就叫做“脏读”——读到了别人没洗干净的数据 ✅🍲😄

🎯 解决办法:

要防止脏读，就必须将隔离级别设置到 Read Committed 及以上:

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

这样，事务只能读取其他事务已提交的数据。

模拟演示:

安排两个终端模拟以下情境:

1. 终端1执行事务A, 修改数据但不提交;
2. 终端2执行事务B, 在不同隔离级别下读取数据;
3. 比较在 READ UNCOMMITTED 和 READ COMMITTED 下的行为差异。

🎓 课堂总结:

“脏读就像看见了别人草稿纸上的答案，结果他最后擦掉改了，你却把旧答案当真了。”

不可重复读 (Non-repeatable Read)

💡不可重复读定义:

不可重复读是指：在一个事务中，对同一条记录的两次读取结果不一致，因为在这两次读取之间，另一事务提交了对该记录的修改。

✅ 案例解释：用户余额查询

👛 业务背景：

用户小王正在查看银行账户余额。数据库中账户信息如下：

账户ID	用户名	余额
101	小王	1000

两个事务并发执行:

事务A（用户小王）:

```
BEGIN;  
SELECT balance FROM account WHERE id = 101;  -- 返回: 1000  
-- ...假设过了一会儿, 再次查看...  
SELECT balance FROM account WHERE id = 101;  -- 结果是?  
COMMIT;
```

事务B（系统后台自动扣款）:

```
BEGIN;  
UPDATE account SET balance = balance - 200 WHERE id = 101;  
COMMIT;
```

结果:

- 如果数据库隔离级别为 Read Committed (读已提交),
- 那么事务A的第一次读取为1000, 第二次读取为800
- 同一事务中, 对同一行记录, 两次读到的内容不一致!

这就是不可重复读现象。

✗问题总结:

- 事务A想“重复读取”某行记录，却因为事务B的更新，读到了不同的结果
- 数据变化对事务A“暴露”了，影响了稳定性和分析的正确性

🛡️如何避免不可重复读?

使用更高的隔离级别，比如：

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

在该级别下，事务A读取的行会被“锁住”，其他事务在提交修改前必须等待，从而保证两次读取结果一致。

🎯 类比总结:

“你查了一次银行余额是1000元，几秒后再查，只剩800了——而你根本没操作过。不是银行错了，而是你没有开‘事务防偷窥模式’。”

模拟演示：

安排实验如下：

1. 终端1开启事务A，读取某行数据；
2. 终端2执行事务B，更新同一行并提交；
3. 终端1再次读取，观察值是否发生变化；
4. 修改隔离级别为 REPEATABLE READ 重新实验，观察现象是否还会发生。

幻读（Phantom Read）

💡 幻读（Phantom Read）定义：

幻读指的是：在同一个事务中，相同的查询条件在两次执行时，读到了不同数量的“行”。
原因是：另一事务在中间插入了新数据，导致查询“看到幻影”一样的新行。

✅ 案例解释：员工表统计人数

👛 业务背景：

某公司 HR 要统计技术部（`department = '技术'`）有多少名员工。

数据库初始如下：

id	name	department
1	张三	技术
2	李四	技术

👤两个事务并发执行:

■事务A（HR 查询统计）:

```
BEGIN;  
SELECT COUNT(*) FROM employee WHERE department = '技术';  
-- 返回: 2  
-- 等待中.....  
SELECT COUNT(*) FROM employee WHERE department = '技术';  
-- 结果是?  
COMMIT;
```

■事务B（另一个用户新增员工）:

```
BEGIN;  
INSERT INTO employee (id, name, department) VALUES (3, '王五', '技术');  
COMMIT;
```

结果:

- 如果使用的是 REPEATABLE READ 隔离级别，行级锁不能阻止新增行
- 所以事务A 第二次查询会返回 3 条记录
- 对于事务A来说，第一次只看到 2 条，第二次突然冒出一条“幻影”行，这就是幻读

问题本质:

- 不是同一行内容变了（那是不可重复读），而是行数变了
- “查询结果行的集合发生了变化”

如何防止幻读?

使用最高隔离级别：SERIALIZABLE，它通过**范围锁（predicate lock）**阻止其他事务插入满足当前事务查询条件的“新行”。

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

课堂总结:

“你查教室有几个人，查完准备点名，刚开口前有人又悄悄走进来。你再一看——人数变了。你不可能知道什么时候‘多出来的’，就像看到了一道幻影。”

演示模拟:

实验:

1. 用事务A查询某条件的多行结果;
2. 事务B插入满足相同查询条件的新记录;
3. 事务A再次查询, 比较前后差异;
4. 改用 SERIALIZABLE 隔离级别, 观察是否还会出现“幻影”行。

隔离级别与并发问题的关系表：

隔离级别	脏读	不可重复读	幻读
Read Uncommitted	✓	✓	✓
Read Committed	✗	✓	✓
Repeatable Read	✗	✗	✓
Serializable	✗	✗	✗

事务的实现机制

- 日志机制（如 Write-Ahead Logging, WAL）
- 锁机制（Locking）：
 - 排它锁（Exclusive Lock, X锁）
 - 共享锁（Shared Lock, S锁）
 - 意向锁（Intent Locks）
- MVCC（多版本并发控制）：PostgreSQL、MySQL InnoDB 使用的并发控制方式，避免加锁带来的性能问题。

日志机制 (Write-Ahead Logging, WAL)

💡 日志机制定义：

日志机制是数据库系统为了保证事务的原子性与持久性，在对数据库做出变更前，先将操作记录写入日志文件，即“先写日志再执行操作”。

即使系统崩溃，也能通过日志恢复数据。

主要类型有：

- Redo Log (重做日志)：用于重放已提交事务的修改，确保事务持久性 (Durability)
- Undo Log (回滚日志)：用于撤销未提交事务的操作，确保事务原子性 (Atomicity)

✅ 典型案例：银行转账崩溃恢复

👛 场景设定：

用户 A 向用户 B 转账 100 元，涉及如下事务操作：

```
BEGIN;  
UPDATE account SET balance = balance - 100 WHERE id = 'A';  
UPDATE account SET balance = balance + 100 WHERE id = 'B';  
COMMIT;
```

系统崩溃时刻：

假设系统在第二条 SQL 执行后、尚未完成 COMMIT，服务器突然断电。

日志机制的作用

情况一：使用日志机制

数据库在执行每条更新语句前，会先写入日志：

- Undo Log 记录 A 和 B 原始余额
- Redo Log 记录 A 扣款后、B 加款后的新值
- 只有当日志写入成功并持久化后，事务才会提交

系统崩溃后：

- 数据库检查事务状态，发现尚未提交
- 使用 Undo Log 回滚操作，使账户恢复原状，避免“钱丢了”或“凭空多出”问题

情况二：没有日志机制

- 系统崩溃时，可能 A 已扣款但 B 未到账
- 事务状态不明，数据不一致，且无法恢复
- 银行系统出现严重漏洞

📦 常见日志机制（以 MySQL InnoDB 为例）：

类型	作用	使用场景
Redo Log	恢复已提交事务数据	崩溃后重放，保障持久性
Undo Log	回滚未提交事务数据	回滚事务，保障原子性与一致性
Binlog	记录所有变更操作	主从复制、归档、审计等

🎯 类比总结:

“日志就像飞行记录仪，即使飞机坠毁，我们也能找回飞行轨迹。数据库也是一样，有了日志，即使系统宕机，数据也能‘飞回来’。”

 演示模拟:

实验:

1. 查看数据库 redo 和 undo 日志文件位置;
2. 在开启事务后断电重启, 观察数据恢复情况;
3. 比较日志写入前后的事务提交行为。

数据库锁机制（ Locking Mechanism ）

🎯一、锁机制定义

锁机制是数据库管理系统用于协调多个事务并发访问数据的手段。它通过加锁/解锁操作，防止数据出现“读写冲突”或“更新丢失”，保障事务的隔离性与一致性。

🔑二、常见锁类型

锁类型	作用	举例
共享锁（S锁）	允许多个事务同时读取	SELECT ... LOCK IN SHARE MODE
排它锁（X锁）	只允许一个事务读取和写入	UPDATE / DELETE 操作时隐式加
意向锁	InnoDB 自动加，用于表级锁兼容	IS（意向共享）、IX（意向排它）
行级锁	锁住某一行数据	并发性能高，开销小
表级锁	锁整个表	并发性能低，简单可靠

三、案例解释：防止库存超卖

业务背景：

某电商平台同时有多个用户抢购手机，库存只有 1 台。库存表如下：

商品ID	商品名	库存
101	手机X	1

两个事务几乎同时发起购买操作。

■ 事务A：用户A购买手机

```
BEGIN;  
-- 查询库存 (加共享锁)  
SELECT stock FROM product WHERE id = 101 FOR UPDATE;  
-- 查询后加排它锁准备更新  
UPDATE product SET stock = stock - 1 WHERE id = 101;  
-- 提交事务  
COMMIT;
```

■ 事务B：用户B同时抢购

```
BEGIN;  
SELECT stock FROM product WHERE id = 101 FOR UPDATE; -- 被阻塞!  
UPDATE product SET stock = stock - 1 WHERE id = 101;  
COMMIT;
```


发生了什么？

- 事务A执行时，给该行数据加了排它锁
- 事务B尝试对同一行加锁时被阻塞，直到A提交
- 最终：只有一个事务能成功购买，另一个等锁释放后再判断库存是否为0，避免超卖

类比总结：

“锁机制就像银行排队取钱，窗口一次只能服务一个人。如果没有锁，两个柜员可能会同时操作同一个账户，就会出错。”

模拟演示：

两个终端执行以下实验：

1. 一个终端执行事务A（更新前不提交）；
2. 另一终端执行事务B（等待中）；
3. 观察锁阻塞现象；
4. 使用 `SHOW ENGINE INNODB STATUS` 查看锁等待队列；
5. 修改 SQL 使用共享锁 `LOCK IN SHARE MODE`，观察读操作是否仍被阻塞。

排它锁（X锁）、共享锁（S锁）、意向锁（Intent Locks）

一、三种锁的概念简介

锁类型	作用	是否阻塞其他事务
共享锁（S锁）	允许多个事务同时读取数据，但不能写入	阻塞写
排它锁（X锁）	只允许一个事务读/写数据，禁止其他事务读写	阻塞读写
意向锁（IS/IX）	表级锁，用于表示事务意图对某些行加S锁或X锁，不阻塞记录操作	用于协调表锁与行锁

二、案例解释与对比

 场景设定：订单表 orders

```
CREATE TABLE orders (  
  id INT PRIMARY KEY,  
  status VARCHAR(20)  
);
```

初始数据：

id	status
1	pending
2	shipped

◆ 1. 共享锁（S锁）案例：两个用户同时查看订单

👤 事务A:

```
START TRANSACTION;  
SELECT * FROM orders WHERE id = 1 LOCK IN SHARE MODE;  
-- 加S锁, 可以读, 不能写
```

👤 事务B:

```
START TRANSACTION;  
SELECT * FROM orders WHERE id = 1 LOCK IN SHARE MODE; -- ✅成功  
UPDATE orders SET status = 'shipped' WHERE id = 1; -- ❌被阻塞 (S锁禁止写)
```

✅ 说明:

- 多个事务可加S锁，共享读取；
- 一旦有S锁，其他事务不能加X锁进行写入；

◆ 2. 排它锁（X锁）案例：一个用户修改订单

👤 事务A:

```
START TRANSACTION;  
UPDATE orders SET status = 'cancelled' WHERE id = 1;  
-- 加X锁
```

👤 事务B:

```
START TRANSACTION;  
SELECT * FROM orders WHERE id = 1; -- ✗被阻塞  
UPDATE orders SET status = 'shipped' WHERE id = 1; -- ✗被阻塞
```

✅ 说明:

- 事务A加了排它锁，只有它能读写；
- 事务B连读都要等待；

◆ 3. 意向锁（Intent Lock）案例：表锁与行锁共存场景

意向锁是InnoDB自动加的表级锁，用来标记事务意图对某行加锁，从而允许表级锁与行级锁协调工作。

示例：两个事务分别加行锁与表锁

💻 事务A（行锁）：

```
START TRANSACTION;  
SELECT * FROM orders WHERE id = 1 FOR UPDATE;  
-- 自动加 IX 意向排它锁（表级） + X锁（行级）
```

💻 事务B（尝试加表级共享锁）：

```
LOCK TABLE orders READ;  
-- ❌ 被阻塞，因为表上已有 IX 锁，冲突！
```

✅ 说明：

- 事务A加行级排它锁前，数据库自动在表上加了 IX（意向排它锁）；
- 事务B试图对整张表加 READ（共享表锁）时，检查发现锁冲突；
- 避免表锁与行锁同时操作导致冲突或死锁。

三、锁兼容性对照表

当前锁类型	请求S锁	请求X锁	请求IS	请求IX
S锁	✓	✗	✓	✗
X锁	✗	✗	✗	✗
IS锁	✓	✗	✓	✓
IX锁	✗	✗	✓	✓

🎓 类比总结:

- S锁 像图书馆里大家可以同时借阅一本书，只能看不能改。
- X锁 就像某人借走一本书去修改，别人不能读也不能改。
- 意向锁 像门口贴出“我要借这本书”的提示，方便别人知道是否可以借整层书架。

MVCC (多版本并发控制, Multi-Version Concurrency Control)

一、MVCC 概念解释

MVCC 是一种让多个事务并发访问数据而互不干扰的并发控制机制。它通过为每条记录保存多个版本（快照），从而避免读写冲突，实现非阻塞的“读写分离”。

MVCC 的核心思想：

- 读操作 读的是数据的某个历史版本（快照）；
- 写操作 创建新的数据版本，不影响当前的读事务；
- 数据记录包含两个隐藏字段：
- 创建版本号（指示哪个事务创建）
- 删除版本号（指示哪个事务删除）

二、案例解释：防止读写冲突的“快照读”

 表结构：用户表 user_balance

用户ID	姓名	余额
1	Alice	100

👤事务A：执行转账，修改余额

```
START TRANSACTION;  
UPDATE user_balance SET balance = balance - 50 WHERE id = 1;  
-- 不提交事务A，持有修改中的新版本，但还没覆盖原记录
```

👤事务B：读取余额

```
START TRANSACTION;  
SELECT balance FROM user_balance WHERE id = 1;  
-- 事务B看到的是事务A修改前的旧版本: balance = 100
```

✓现象分析:

- 事务A 修改了余额，但没提交，这时创建了一个“新版本”；
- 事务B 启动后，看到的是自己启动时刻的快照版本；
- 事务B不会被事务A阻塞，读取的是旧数据，保持一致性；
- 事务A提交后，旧版本可能被清理。

🎓 类比总结：

“MVCC就像你在看一份快照账本，别人修改了原账本内容，你仍然按照你拿到时的状态处理工作，互不干扰。”

三、MVCC应用于 InnoDB: 快照读 vs 当前读

操作方式	是否使用MVCC	是否加锁	场景举例
SELECT * FROM table	✅ 是	❌ 否	快照读，非锁定读，常用于查询
SELECT * FROM table FOR UPDATE	❌ 否	✅ 是	当前读，锁定数据用于修改
UPDATE、DELETE	❌ 否	✅ 是	当前读，需写锁

模拟实验：

在两个终端执行以下步骤观察 MVCC：

1. 事务A开启后执行 UPDATE 但不提交；
2. 事务B随后开启执行普通 SELECT；
3. 观察事务B读到的是修改前的数据；
4. 用 SELECT ... FOR UPDATE 测试“当前读”行为，观察被阻塞情况；
5. 再讲解版本号字段在 InnoDB 中的原理（如隐藏字段、undo log）。

总结：MVCC 优势

-  高并发读写：避免读操作被写操作阻塞；
-  性能更优：无需加锁即可读；
-  事务隔离性保障：支持快照读；
-  实现 READ COMMITTED 和 REPEATABLE READ 隔离级别。

两段锁协议 (Two-Phase Locking, 2PL)

一、什么是两段锁协议 (2PL) ?

两段锁协议 (2PL) 是一种保证事务**遵守隔离性 (Isolation) **的加锁机制。其基本思想是：

- 将事务的锁操作分为 两个阶段：
 - i. 加锁阶段 (Growing Phase) : 事务可以不断加锁，但不能解锁；
 - ii. 解锁阶段 (Shrinking Phase) : 事务可以释放锁，但不能再加新锁。

一旦进入解锁阶段，加锁操作必须停止。

两段锁协议（Two-Phase Locking, 2PL）的核心作用是：

✅ 确保事务的隔离性和可串行化（Serializability）

也就是说，在并发执行多个事务的环境中，通过控制加锁与解锁的顺序，使得数据库执行结果就像是所有事务一个接一个串行执行的一样，从而避免并发引发的异常问题，如：

- 脏读（Dirty Read）
- 不可重复读（Non-repeatable Read）
- 幻读（Phantom Read）
- 丢失更新（Lost Update）等

🔍 两段锁协议的作用详解

作用	说明
🎯 实现串行化调度	确保多个事务执行的结果与某种串行顺序相同，是事务正确执行的基础。
🔑 统一加锁与解锁流程	避免在事务过程中反复加锁解锁引发冲突，提高事务的可预测性和一致性。
🧱 支持隔离级别的实现	是多数关系型数据库（如 MySQL InnoDB）实现事务隔离机制的底层原理。
⚠️ 避免并发异常	防止两个或多个事务并发操作同一资源时引起数据错误。
💡 为严格两段锁协议提供基础	严格 2PL 是防止“脏读”和“丢失更新”的常用策略。

知识要点

- “两段”并不是两把锁，而是两个阶段（先加锁，后解锁）。
- 一旦开始释放锁，就不能再加任何新锁。
- 满足 2PL 的事务调度都是可串行化的（即不会出错的调度）。

总结：

两段锁协议的作用是通过限制锁的加解顺序，确保并发事务的执行结果与某种串行顺序等价，从而维护事务的隔离性和数据一致性。

🎓 二、案例解释：两个事务对账户表进行操作

我们使用一个简单的账户表 `account(id, name, balance)`，进行以下两个并发事务操作。

场景描述:

事务 A (T1) : 将账户 A 扣 100 元, 转账给 B

```
START TRANSACTION;
-- 加锁阶段
SELECT balance FROM account WHERE name = 'A' FOR UPDATE;
UPDATE account SET balance = balance - 100 WHERE name = 'A';

SELECT balance FROM account WHERE name = 'B' FOR UPDATE;
UPDATE account SET balance = balance + 100 WHERE name = 'B';
-- 解锁阶段 (自动释放)
COMMIT;
```

事务 B (T2) : 统计账户 A 和 B 的总余额

```
START TRANSACTION;
SELECT SUM(balance) FROM account WHERE name IN ('A', 'B');
COMMIT;
```

🔍 分析两段锁机制的作用：

- 在 2PL 协议下，T1 在“加锁阶段”获得了 A 和 B 两条记录的排它锁；
- T2 无法读取这两条记录（因为未提交的事务加了排它锁）；
- T2 必须等待 T1 提交并释放锁后才能读取，防止读取到转账中间状态；
- 加锁操作都发生在事务开始阶段，后续只释放锁而不再加锁，满足 2PL 的规则。

✅ 结果与优点

- 避免了 脏读、不可重复读、幻读等并发问题;
- 确保事务执行具有 串行等效性 (serializability);
- 实现了事务间的严格隔离。

延伸：严格两段锁协议（Strict 2PL）

在标准2PL中，只要遵守“先加后解”的规则即可。而在 严格2PL中：

- 所有 排它锁 必须 直到事务提交或回滚时 才释放；
- 可以更强地保证不会产生 脏读或丢失更新。

事务可串行化 (Serializability)

✅ 一、什么是事务可串行化?

事务的可串行化 是指：多个并发事务的执行结果，与某种**串行（一个一个顺序执行）**执行它们的结果是一样的。

它是事务调度的正确性最高级别保证，也是事务并发控制的目标。

🎓 二、案例解释：账户转账与余额查询

设有一个银行账户表 `account(name, balance)`，有两个事务：

👛 事务 A (T1)：将张三的账户减 100 元，转给李四

```
START TRANSACTION;  
UPDATE account SET balance = balance - 100 WHERE name = '张三';  
UPDATE account SET balance = balance + 100 WHERE name = '李四';  
COMMIT;
```

🔍 事务 B (T2)：查询张三与李四的总余额

```
START TRANSACTION;  
SELECT SUM(balance) FROM account WHERE name IN ('张三', '李四');  
COMMIT;
```

三、并发执行 vs 串行执行

✗ 非串行执行（并发）风险示意：

如果 T2 恰好在 T1 的转账“中间”执行（张三扣了钱，李四还没加），它会读到：

- 张三：余额减少了
- 李四：余额还未增加

查询结果会出现错误的总余额，例如比真实少 100 元 —— 这就不是一个“串行”执行结果，会违反一致性。

✓ 可串行化执行效果:

如果数据库使用串行化机制调度事务，如：

- 先执行完 T1（完成转账），再执行 T2（读总余额）
或
- 先执行完 T2（读原始余额），再执行 T1（转账）

那么，T2 看到的数据就是某个事务完成前或后的完整数据状态，结果是正确的。这种情况就称为“可串行化”。

 四、数据库如何实现串行化?

通常通过以下技术实现:

技术	说明
两段锁协议 (2PL)	控制加锁解锁顺序，避免交叉执行引发错误。
MVCC (多版本并发控制)	保证读写互不干扰，提供一致视图。
SERIALIZABLE 隔离级别	最强的隔离级别，通常会锁定整张表或范围，确保严格串行效果。

一句话总结:

事务可串行化的目的是确保并发事务的执行结果与某种串行顺序执行的结果完全一致，从而保障数据库的一致性和正确性。

死锁 (Deadlock)

✓ 一、什么是死锁?

死锁 (Deadlock) 是指两个或多个事务在执行过程中, 因争夺资源 (如锁) 而形成一种互相等待的状态, 导致都无法继续执行。

🔍 二、产生死锁的四个必要条件:

条件	解释
互斥条件	某资源在一时刻只能被一个事务使用 (如一个行锁只能被一个事务持有)。
占有且等待	事务已经占有资源, 同时又在等待其它资源 (锁)。
非抢占条件	事务占有的资源不能被其他事务强行抢占, 只能等其主动释放。
循环等待	存在一组事务 $\{T_1, T_2, \dots, T_n\}$, T_1 等待 T_2 持有的资源, T_2 等待 T_3最后 T_n 又等待 T_1 , 形成环路。

💡 只要这四个条件同时满足, 系统就可能产生死锁。

三、典型死锁案例

 事务A:

```
BEGIN;  
UPDATE account SET balance = balance - 100 WHERE name = '张三';  -- 锁住张三  
-- 等待锁住李四
```

 事务B:

```
BEGIN;  
UPDATE account SET balance = balance + 100 WHERE name = '李四';  -- 锁住李四  
-- 等待锁住张三
```

两个事务互相等待，就陷入了死锁，数据库系统需要介入解决。

四、死锁的处理策略

1. 死锁检测 (Deadlock Detection)

- 数据库系统运行时维护锁等待图;
- 定期检测是否形成循环等待;
- 若发现死锁, 强制回滚其中一个事务以破局;
- MySQL InnoDB默认采用该策略, 一旦检测到死锁, 就报错并回滚一个事务。

2. 死锁预防 (Deadlock Prevention)

- 在事务执行前或执行中进行资源排序;
- 通过某种顺序保证不会形成循环等待;
- 例如: 要求所有事务必须按相同顺序请求资源 (例如按账户名字典序加锁);

3. 死锁避免 (Deadlock Avoidance) (较少使用)

- 通过预测可能产生死锁的调度路径, 提前拒绝某些资源分配;
- 实现复杂, 开销大, 实际应用不多。

五、死锁的检测与定位（MySQL 举例）

SHOW ENGINE INNODB STATUS;

查看 InnoDB 的死锁日志，可以看到：

- 哪些事务参与了死锁；
- 哪个事务被回滚；
- 涉及哪些表和记录。

六、教学扩展：如何避免死锁的编程建议

- 避免长事务，减少锁持有时间；
- 所有事务按照相同顺序访问资源；
- 尽量一次性加锁完所有需要的数据行，不要边处理边加锁；
- 使用 `SELECT ... FOR UPDATE` 明确加锁范围；
- 在并发高场景下，考虑使用更低隔离级别结合业务容错。

七、总结

死锁是由多个事务对资源交叉加锁、互相等待而导致的系统停滞，合理的加锁策略和数据库机制可避免或化解死锁。

SQL 中的事务操作

- BEGIN TRANSACTION / START TRANSACTION
- COMMIT
- ROLLBACK
- SAVEPOINT 与 ROLLBACK TO SAVEPOINT（部分回滚）

事务四大特性（ACID：原子性Atomicity、一致性Consistency、隔离性Isolation、持久性Durability）课程测验

✅ 单项选择题（每题只有一个正确答案）

1. 以下哪项最能准确描述事务的“原子性”?
- A. 事务执行必须按照顺序进行
 - B. 事务中的操作要么全部执行, 要么全部不执行
 - C. 事务可以部分回滚部分提交
 - D. 事务必须每次只处理一个数据项

正确答案: B

2. 如果系统崩溃后，已提交的事务修改仍然保留，这体现了事务的哪种特性？

- A. 原子性
- B. 一致性
- C. 隔离性
- D. 持久性

正确答案：D

3. 以下哪种情况最能体现事务的一致性特性?

- A. 一个事务提交后, 其修改立即对其他用户可见
- B. 一个事务执行过程中始终读到相同版本的数据
- C. 数据在事务开始和结束时都满足数据库的完整性约束
- D. 多个事务可以并发执行

正确答案: C

4. 为了避免两个事务在同时修改相同记录时产生冲突，数据库使用哪种技术来实现“隔离性”？

- A. 异常捕获
- B. 日志机制
- C. 加锁机制
- D. 数据同步

正确答案：C

5. 如果一个事务失败并回滚，则它对数据库的影响应为：

- A. 保留修改
- B. 修改部分保留
- C. 所有修改全部撤销
- D. 数据库将崩溃

正确答案：C

6. 哪个选项最准确地描述“隔离性”?

- A. 所有事务必须串行执行
- B. 事务不能修改数据库
- C. 一个事务的中间状态对其他事务不可见
- D. 一个事务只能读取自己提交的数据

正确答案: C

7. 以下哪种故障情形最能考察“持久性”?

- A. 并发事务导致脏读
- B. 写入操作导致死锁
- C. 系统断电或崩溃
- D. 不满足主键约束

正确答案: C

8. 若事务在执行过程中违反了约束（如唯一性），该事务应：

- A. 忽略错误并继续执行
- B. 提交后修复错误
- C. 部分提交
- D. 回滚事务，保持一致性

正确答案：D

9. 以下哪项不是事务四大特性之一?

- A. 并发性
- B. 原子性
- C. 一致性
- D. 持久性

正确答案: A

10. 下列哪个隔离级别最容易导致幻读?

- A. READ UNCOMMITTED
- B. READ COMMITTED
- C. REPEATABLE READ
- D. SERIALIZABLE

正确答案: B

事务异常现象（如脏读、不可重复读、幻读等）课堂测验

✓ 单项选择题：事务异常相关

1. 事务T1读取了事务T2尚未提交的数据，这种现象称为：

- A. 幻读
- B. 不可重复读
- C. 脏读
- D. 串行化

✓ 正确答案：C

2. 脏读最可能在以下哪种隔离级别中发生?

A. SERIALIZABLE

B. REPEATABLE READ

C. READ COMMITTED

D. READ UNCOMMITTED

✓ 正确答案: D

3. 在一个事务中两次读取同一条记录，结果却不同，这种现象称为：

- A. 脏读
- B. 不可重复读
- C. 幻读
- D. 回滚失败

✓ 正确答案：B

4. 事务T1在读取某个范围内的记录时，事务T2插入了一条新记录，导致T1再次读取结果不同。这属于哪种异常？

- A. 脏读
- B. 不可重复读
- C. 幻读
- D. 死锁

✓ 正确答案：C

5. 幻读通常发生在哪种类型的 SQL 操作中?

- A. 单条记录更新
- B. 聚合函数运算
- C. 范围查询或聚合查询
- D. 精确主键查找

✓ 正确答案: C

6. 哪种隔离级别能够防止幻读?

A. READ COMMITTED

B. READ UNCOMMITTED

C. REPEATABLE READ (MySQL默认)

D. SERIALIZABLE

✓ 正确答案: D

7. 在 READ COMMITTED 隔离级别下, 可以避免以下哪种事务异常?

- A. 幻读
- B. 不可重复读
- C. 脏读
- D. 死锁

✓ 正确答案: C

8. 若两个事务并发执行，均试图对同一数据进行更新，会造成以下哪类问题？

- A. 幻读
- B. 丢失更新
- C. 脏读
- D. 不可重复读

✓ 正确答案：B

9. 在数据库中，为避免不可重复读，可以采用哪种策略？

- A. 使用MVCC
- B. 禁用自动提交
- C. 提高硬件性能
- D. 降低隔离级别

✓ 正确答案：A

10. 如果一个事务看到另一个事务插入的新记录，并且这些记录在它第一次读取时还不存在，则发生了：

- A. 幻读
- B. 脏读
- C. 死锁
- D. 丢失更新

✓ 正确答案：A